TECHNICAL DESIGN DOCUMENT

# NESTORA

Smart township management for a gated Pune community — MySQL persistence and Groq-assisted complaint triage.

| | |
|---|---|
| Version | 2.0 (database migration & AI integration) |
| Date | 4 November 2026 |
| Scope | Architecture, schema, flows, AI workflow, safety handling, security, testing |
| Status | Implemented; MySQL and live-Groq paths not exercised in the build sandbox (see §11) |

## Contents


1. Overview and goals
2. Architecture and technology decisions
3. Database schema
4. Main user flows
5. AI classification workflow
6. Fallback and emergency handling
7. UI / UX decisions
8. Security and privacy
9. Testing
10. Assumptions
11. Limitations and future improvements

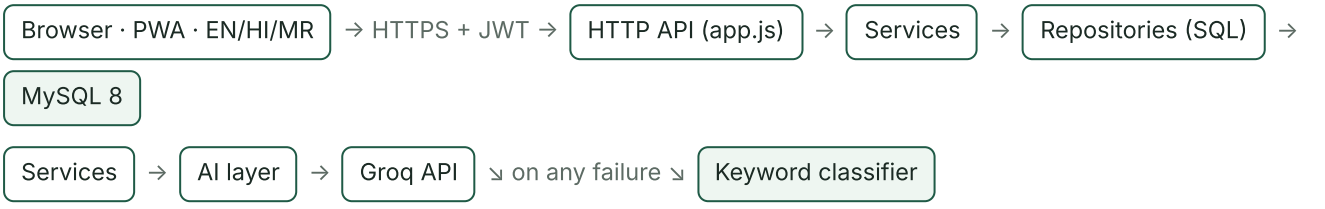

## 1. Overview and goals

NESTORA replaces Excel maintenance bills, manual payment matching, WhatsApp complaint groups and paper visitor registers for a township of 2,400 apartments in 28 towers served by 180 staff. Monthly maintenance is **flat area × ₹6.50**.

This revision (a) replaces the JSON-file store with MySQL, (b) replaces the keyword-only complaint classifier with an LLM classifier (Groq) that is validated, bounded and always backed by a deterministic fallback, and (c) keeps the existing UI, branding and workflows intact.

| Goal | How it is met |
|---|---|
| Relational persistence | 26 normalised InnoDB tables, foreign keys, indexes, CHECK and UNIQUE constraints (§3) |
| Money safety | One transaction per state-changing request; database-level uniqueness for payment references, receipts, bank transactions and UTRs |
| Trustworthy AI | Schema-validated output, one safe retry, keyword fallback, human review queue, audit of corrections (§5–6) |
| Emergency safety | Rule engine independent of the model; the model can raise but never lower urgency (§6) |

## 2. Architecture and technology decisions




| Decision | Choice | Reason |
|---|---|---|
| Runtime | Node.js 22, ES modules, no framework | Existing codebase and 40 passing tests; avoids a rewrite. Built-in `fetch` and `node:test` keep dependencies to one ( `mysql2` ). |
| Database | MySQL 8, InnoDB, utf8mb4 | Required by the brief; transactional, foreign keys, CHECK constraints (8.0.16+). |
| Data access | Repository layer with parameterised SQL; no ORM | All SQL in `server/repos` ; services never build SQL; injection-safe by construction. |

| | | |
|---|---|---|
| Engine abstraction | `server/db` adapter: MySQL (production), SQLite via `node:sqlite` (tests, `dev:sqlite`) | The same `schema.sql` runs on both so tests exercise real constraints and transactions where MySQL is unavailable. SQLite is never selected unless explicitly requested. |
| Identifiers | Readable ids (`CMP-0001`, `BILL-0001`) from a `counters` table | Preserves existing ids and receipts; the counter row is locked for the transaction so ids are gap-free and race-safe. |
| Time / money | UTC `DATETIME(3)`, `DATE`, `DECIMAL(12,2)` | No floating-point money or timezone drift (`dateStrings`, `decimalNumbers`). |
| Auth | HS256 JWT, role claims, demo OTP | Mock OTP per the brief; roles enforced per route. |
| AI provider | Groq Chat Completions, JSON mode, temperature 0 | Low latency, structured output; model is configurable (`GROQ_MODEL`) because hosted model catalogues change. |

> **Stack note.** An earlier requirements document named Spring Boot and React. The delivered, working system is Node and vanilla-JS; this upgrade keeps it rather than rewriting working features. The database, API contracts and layering map one-to-one to a Spring Data JPA port should that be required.

## 3. Database schema

Created by `database/schema.sql` (idempotent, recorded in `schema_migrations`). Sample data is loaded by `scripts/db.js seed`; legacy JSON is imported by `import-json` (taxonomy mapped Housekeeping→Cleaning, Critical→Emergency).

| Group | Tables | Notes |
|---|---|---|
| Reference | `roles`, `app_settings`, `counters`, `schema_migrations` | Rate, due days and late-fee policy live in `app_settings` — configurable, never hard-coded. |
| People & property | `flats`, `flat_upi_ids`, `users`, `staff` | UPI id → flat is UNIQUE; phone UNIQUE; user→role and user→flat FKs. |
| Billing | `maintenance_bills`, `flat_ledger`, `payment_sessions` | One bill per (flat, period). Ledger rows are bills, late fees and payments. |
| Reconciliation | `bank_statements`, `bank_transactions` | Status ∈ matched, partial, unmatched, needs_review, duplicate; reasons and confidence stored. |
| Complaints | `complaints`, `complaint_photos`, `complaint_history`, `complaint_corrections`, `complaint_ratings` | Classification result, method, model, confidence, explanation, review flag and safety rules stored on the complaint; self-FK `linked_to` for duplicates. |
| Community | `visitors`, `guest_codes`, `notices`, `amenities`, `bookings`, `enquiries` | `visitors.client_id` UNIQUE makes offline replay idempotent; `bookings.slot_key` UNIQUE prevents double booking. |
| Governance | `notifications`, `audit_logs` | Append-only trail of financial, reconciliation, correction and retention events. |

### Constraints that protect money

| Constraint | Prevents |
|---|---|
| `flat_ledger.payment_ref` UNIQUE (normalised) | Posting the same payment reference or UTR twice |
| `flat_ledger.receipt_no` UNIQUE · `flat_ledger.bank_txn_id` UNIQUE | Duplicate receipts; one bank transaction credited to two ledgers |
| `bank_transactions.utr_key` UNIQUE (set only on the canonical row) | Two live rows for one UTR. Re-imports are stored with status `duplicate` and a NULL key. |
| `maintenance_bills (flat_no, bill_period)` UNIQUE | Double billing a month |
| CHECK: amount > 0, enums for status/category/priority/method, confidence 0–1 | Invalid values reaching storage regardless of application bugs |

Indexes cover the hot paths: ledger by (flat, date) and (kind, date); complaints by status, (category, priority), (assignee, status), created time and review flag; bank transactions by status, fingerprint and flat; audit by time and action.

## 4. Main user flows

### 4.1 Resident pays maintenance

Sign in (OTP) → Dashboard: dues → Pay (test mode) → Checkout session → Complete (compare-and-set) → Ledger + receipt in one transaction

Partial payments are allowed; payments settle charges oldest-first. A completed session cannot be completed again.

### 4.2 Bank statement reconciliation

Upload CSV → Parse & validate → Duplicate check (fingerprint + UTR) → Match: reference, UPI, amount → Matched / Partial / Needs review / Unmatched → Human approves → Ledger posting

- Exact amount alone never confirms a match. UPI + amount without a payment reference is *needs review*.
- Upload never posts. Only exact reference matches may be auto-posted, and only by an explicit staff action; every approval, rejection and auto-post is audited.

### 4.3 Complaint lifecycle

Resident submits → Classify (§5) → Auto-suggest staff → Emergency? duty manager → Duplicate link → Help desk: correct / assign / status → Resolved by a person → Rate or reopen

### 4.4 Other flows

- **Gate:** guard registers a visitor (queued locally if offline, replayed idempotently by `clientId`) → resident approves or denies → guard records entry/exit. Photos are deleted after 90 days.
- **Committee:** collections, ageing, resolution time and reopen % computed from ledger and complaint rows; monthly CSV export.
- **Amenities:** slot availability, one live booking per slot (database constraint), 24-hour cancellation rule.

## 5. AI classification workflow

1 Safety rules → 2 Redact & build prompt → 3 Groq (JSON, temp 0, timeout) → 4 Validate → 5 Merge with rules → 6 Store + method

| Step | Detail |
|---|---|
| Input | Title and description only. Phone numbers, e-mail addresses, ID numbers, resident and flat identifiers are removed or never sent. The staff roster is sent as id, team and current load. |
| Output (required JSON) | `category` (Plumbing, Electrical, Lift, Security, Cleaning, Civil, Other), `priority` (Low, Medium, High, Emergency), `summary`, `suggested_department`, `suggested_staff`, `confidence` (0–1), `explanation` |
| Validation | Strict: enum membership, numeric range, bounded text, staff must be on duty in the chosen department. Anything else is rejected — the model's text is never stored unchecked. |
| Retry | An invalid answer is retried once with the validation errors fed back; a model that rejects JSON mode is retried once without it. A second failure ⇒ fallback. |
| Stored | `ai_category`, `ai_priority`, `ai_confidence`, `ai_explanation`, `classification_method` (`groq_ai` \| `keyword_fallback` \| `manual`), `classification_model`, `suggested_department` |
| Review | Confidence < `AI_MIN_CONFIDENCE` (0.6), AI/keyword disagreement, or a resident category the AI contradicts ⇒ `needs_review` with reasons; shown in the help-desk review queue. |
| Correction | Help desk changes category or priority in one action → method becomes `manual`, a row is added to `complaint_corrections` and `audit_logs`, history is appended. |

## 6. Fallback and emergency handling

### 6.1 Failure matrix

| Condition | Behaviour | Complaint |
|---|---|---|
| No `GROQ_API_KEY` | Mock/fallback mode: keyword classifier, zero network calls | Created, `keyword_fallback` |
| Network error, timeout, 401, 429, 5xx, empty body | Keyword fallback; error recorded without key or complaint text | Created, `keyword_fallback` |
| Malformed or invalid JSON / unknown category | One retry, then fallback; bad data never stored | Created, `keyword_fallback` |

| | | |
|---|---|---|
| Local call budget exceeded ( `AI_MAX_CALLS_PER_MIN` ) | Fallback without calling the API | Created |
| Low confidence | Kept, flagged for manual review | Review queue |

### 6.2 Emergency rule (independent of the AI)

Regular-expression rules (English, Hindi, Marathi) detect **fire, gas leak, lift stuck/trapped, electric shock**. They run first and their result is applied after the model:

- Priority is forced to **Emergency**, status becomes **escalated**, the complaint is assigned to the duty manager and a notification is stored.
- The model's verdict is merged one-way: it may raise a complaint to Emergency (escalated too, but correctable) and can never lower a rule-triggered emergency — even if it returns *Low*, errors, or returns junk. Instructions embedded in complaint text (prompt injection) cannot disable this.
- Help desk cannot lower a rule-triggered emergency; only a person resolves it. Complaints are never auto-closed.
- Duplicates are **linked** to the original and stay visible and open.

## 7. UI / UX decisions

- **Preserved:** branding, cinematic landing page, navigation, dashboards, responsive layout, three languages.
- **Added:** in the help-desk complaint drawer — classification method badge (AI / keyword fallback / corrected by staff), confidence and explanation, manual-review banner with a one-tap *Confirm*, priority control, correction audit trail. A safety banner states the escalation was by rule.
- **Residents** never see AI internals (reasons, confidence, review flags).
- **States:** loading skeletons, empty states, success toasts and inline errors on every data screen; the mobile top bar was fixed for narrow phones. Verified at 390, 820 and 1440 px with no horizontal overflow on the help desk.
- Principle: most important information within two taps; every automated decision is visible and reversible by a person.

## 8. Security and privacy

| Area | Control |
|---|---|
| Secrets | `GROQ_API_KEY` , `DB_PASSWORD` , `JWT_SECRET` only in backend environment; `.env` git-ignored; `.env.example` holds names only; a test scans the repository and browser bundle for secrets. |
| AuthN / AuthZ | JWT (HS256, 8 h); every route declares allowed roles; residents are confined to their own flat; matrix test covers each route (403 / 401). |
| Input | Length, enum, date and phone validation; malformed JSON ⇒ 400; 6 MB body cap; photo type and size checks; parameterised SQL only. |
| Rate limiting | Per-IP API and OTP limits, per-user complaint creation, public form limit; 429 with `Retry-After` ; local budget for Groq calls. |

| | |
|---|---|
| Logging | Keys, passwords and complaint text are never logged; database description prints no password. |
| Privacy (DPDP 2023) | Privacy notice on gate screens; minimal data to the AI provider; visitor photos deleted after 90 days by an hourly job (audited); audit trail of staff actions. |
| Headers | `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy`, `Cache-Control: no-store` on the API. |

## 9. Testing

Run with `npm test` (Node's built-in runner). Result of the final run in the build environment: **96 tests — 95 passed, 0 failed, 1 skipped** (the live-Groq test, which needs `GROQ_API_KEY`). Database tests execute `schema.sql` on SQLite; they were **not** run against a MySQL server in the build environment.

| Required area | Coverage (test files) |
|---|---|
| Database connectivity | round-trip query, engine reported, migration recorded, configuration from environment, failure without `DATABASE_URL` (`database.test.js`) |
| Resident and billing persistence | CRUD across independent repository instances; brief sample data reproduced from rows; bill generation idempotent (`database.test.js`, `billing.test.js`) |
| Payment matching | outcomes on the sample file; UPI+amount ⇒ review; overpayment ⇒ review; import never posts (`matching.test.js`) |
| Duplicate UTR / payment reference | blocked in code and by unique keys; same bank transaction cannot post twice; concurrent postings ⇒ exactly one commit; atomic rollback (`matching.test.js`, `billing.test.js`, `database.test.js`) |
| AI classification | valid answer stored with method and model; request hygiene (no PII, key only in header) (`ai.test.js`) |
| Invalid AI response | validator rejects each malformed field; retry with feedback; two failures ⇒ fallback; unknown category never stored (`ai.test.js`) |
| API failure fallback | network, timeout, 401, 429, 500, empty body; no key; call budget (`ai.test.js`) |
| Emergency escalation | fire/gas/lift/shock stay Emergency even if the model says Low, fails or returns junk; prompt injection; cannot be downgraded; Hindi (`ai.test.js`, `triage.test.js`) |
| Low-confidence review | flagged with reason; queue; confirm or correct with audit (`ai.test.js`) |
| Role-based authorisation | route × role matrix, cross-flat isolation, JWT tamper/expiry (`security.test.js`, `api.test.js`) |

Browser checks (Playwright/Chromium against the SQLite dev runner): resident sign-in, raising a gas-leak complaint, help-desk escalation banner, classification card, refusal to lower an emergency, and no horizontal overflow at mobile, tablet and desktop widths.

## 10. Assumptions

- MySQL 8.0.16+ (CHECK constraints are enforced) with InnoDB and utf8mb4.
- One township, one currency (INR); late-fee policy is whatever the committee configures — none by default.

- The mock OTP and test-mode payments are intentional for evaluation; no real money or card data is handled.
- Hosted-model catalogues change: the default model is configurable and should be confirmed against Groq's current list.
- The delivered stack (Node, vanilla JS) is retained from the earlier build.

## 11. Limitations and future improvements

| Limitation | Impact / next step |
|---|---|
| **MySQL and live Groq were not exercised in the build sandbox** (registries, MySQL and api.groq.com were unreachable) | The MySQL adapter and the live Groq call are implemented but unverified against real services; SQLite runs the identical schema for tests, and the Groq transport was tested with an injected `fetch`. Run `npm test` with `GROQ_API_KEY` set and smoke-test on MySQL before go-live. |
| Demo OTP and test-mode payments | Replace with an SMS gateway and a PCI-compliant payment gateway. |
| In-memory OTP and rate-limit counters | Move to Redis before running more than one instance. |
| Photos stored in the database | Move to object storage with signed URLs for scale. |
| Analytics read rows into the application | Push aggregation into SQL / materialised views at full 2,400-flat scale. |
| Keyword fallback is English/Hindi/Marathi keywords only | Collect help-desk corrections (already stored) to evaluate accuracy and tune thresholds. |
| No tenant-level schema migrations framework | Adopt versioned migrations (Flyway-style) for later schema changes. |